

Local RAG Chatbot Project Workflow

Local RAG Chatbot Project Workflow

First, I started by setting up the complete Python environment on my system. I installed Python, configured PATH variables, and created a virtual environment using venv. After activating the virtual environment, I installed all required libraries such as LangChain, Streamlit, ChromaDB, Ollama integrations, Pandas, and other dependencies.

After the Python setup, I installed Ollama to run Large Language Models locally on my machine. Then I downloaded two Ollama models:

- llama3.2:3b → used as the chatbot LLM
- mxbai-embed-large → used for embedding generation

This allowed the entire project to run locally without using OpenAI APIs.

Data Collection using Bright Data

After setting up the environment, I moved to the data collection stage.

I used Bright Data Dataset API to scrape Wikipedia articles dynamically using keywords stored inside keywords.xlsx.

To perform the scraping process, I ran:

```
1_scraping_wikipedia.py
```

This script:

- authenticated the Bright Data API
- created a scraping snapshot
- scraped Wikipedia pages
- downloaded the dataset locally

Initially, I faced multiple issues such as:

- "Customer is not active"
- invalid credentials
- dataset not ready errors

To solve this, I contacted Bright Data support and explained that I was building a local RAG AI project. Their support team activated my account and added free credits for testing purposes.

After fixing the API configuration and account issues, the scraping pipeline successfully downloaded the Wikipedia dataset.

Dataset Processing

The scraped data was stored inside:

```
datasets/data.txt
```

The dataset contained multiple JSON records from Wikipedia pages.

After the data was downloaded successfully, I processed the dataset line-by-line using Python.

Chunking + Embedding + Ingestion

Next, I ran:

```
2_chunking_embedding_ingestion.py
```

This script performed three major tasks automatically:

1. Chunking

The large Wikipedia articles were split into smaller chunks using RecursiveCharacterTextSplitter from LangChain.

Chunking configuration:

- chunk size = 1000
- chunk overlap = 200

Chunking is important because large documents cannot efficiently fit into LLM context windows directly.

So after running this script:

- documents were automatically converted into chunks
- overlapping chunks improved retrieval quality

2. Embedding Generation

After chunking, each chunk was converted into vector embeddings using mxbai-embed-large through Ollama.

Embeddings convert text into numerical vector representations that capture semantic meaning.

This enables semantic similarity search later during retrieval.

3. Storing in ChromaDB

The generated embeddings were then stored inside:

```
chroma_db/
```

using ChromaDB.

Each chunk stored:

- content
- source metadata
- title metadata

This created the vector database required for semantic retrieval.

Building the RAG Chatbot

After creating the vector database, I built the complete RAG chatbot pipeline inside:

3_chatbot.py

The workflow of the chatbot is:

User Question
↓
Embedding Generation
↓
Semantic Search in ChromaDB
↓
Relevant Context Retrieval
↓
Prompt Construction
↓
LLM Response Generation
↓
Answer + Sources

When the user asks a question:

1. the question gets converted into embeddings
2. ChromaDB performs semantic similarity search
3. relevant chunks are retrieved
4. retrieved context is passed into the LLM
5. the LLM generates a grounded response using retrieved knowledge

Improving Retrieval Quality

Initially, the chatbot returned unrelated sources sometimes because semantic search was retrieving loosely related chunks.

To improve this:

- I added relevance score filtering
- optimized retrieval count using k=5
- filtered low-quality retrievals
- improved prompt engineering

This significantly improved source relevance and answer quality.

Streamlit Frontend

For the frontend interface, I used Streamlit.

The chatbot interface supports:

- chat UI
- user prompts
- AI-generated responses
- source display
- session history

The chatbot runs locally using:

`http://localhost:8501`

Final Outcome

Finally, I successfully built a fully local Retrieval-Augmented Generation (RAG) chatbot capable of:

- semantic document retrieval
- vector search
- local LLM inference
- context-aware answering
- source attribution

The entire system works locally using:

- Ollama
- LangChain
- ChromaDB
- Streamlit

without depending on cloud-based LLM APIs after setup.